第一阶段：环境部署与 hello 验证

工大喵喵队 2026 年 11 月 7 日

一、阶段任务

1.把香山（XiangShan）和 NEMU 的开发环境搭起来，版本固定下来；

2.跑通 hello 程序，输出 hello xiangshan,I am xxx,IP address；

3.把环境部署和操作过程完整记录下来，后面 vadd 波形、vdot 集成都用这套环境。

二、环境与版本

我们用的是 VMware 里的 Ubuntu 虚拟机，主要配置如下：

项目	配置
宿主机	Windows 11+VMware Workstation
虚拟机	Ubuntu 24.04 LTS，x86_64
内存	19 GiB，swap 8 GiB
磁盘	根分区 118 GB，剩余约 97 GB
虚拟机 IP	192.168.124.133
JDK	GraalVM 21.0，/opt/graalvm-jdk-21
LLVM/Clang	/usr/local/llvm/bin
Verilator	5.052
交叉编译	riscv64-linux-gnu-
波形工具	GTKWave
香山	commit 16ae9ddcda54fc9a2fddffad73174cf793ac7814
NEMU	commit a48a11f208ff81ad765591d83271d1b339fb3861
目标配置	KunminghuV2MinimalConfig
boot flash	16 字节，SHA256 fea54865…9724

香山和 NEMU 都放在~/risv/xs-env/下，测试程序放在~/risv/project/software/，编译产物和波形分开放在~/risv/artifacts/、~/risv/waves/。这样后面重新编译时不会把之前的日志和波形覆盖掉。

三、部署过程

1.拉取代码并固定版本

```
git clone https://github.com/OpenXiangShan/xs-env.git
cd xs-env
# 按 xs-env 的 README 初始化工具链和子模块
cd XiangShan
git checkout 16ae9ddcda54fc9a2fddffad73174cf793ac7814
cd ../NEMU
git checkout a48a11f208ff81ad765591d83271d1b339fb3861
```

固定版本后，构建前先 git status 看一下有没有未记录的修改。后面 vdot 补丁、NEMU 补丁都是在这个基础上打的。

2.设置环境变量

```
export JAVA_HOME=/opt/graalvm-jdk-21
export PATH=$JAVA_HOME/bin:/usr/local/llvm/bin:$PATH
export NOOP_HOME=$HOME/risv/xs-env/XiangShan
export NEMU_HOME=$HOME/risv/xs-env/NEMU
```

这几个变量里，NOOP_HOME 和 NEMU_HOME 后面踩过坑：有一次 KunminghuV2MinimalConfig 构建时报 NoSuchElementException:NOOP_HOME，补上变量后就好了。具体是不是唯一原因，我们没有再做对照实验，所以文档里只写“设置后构建通过”，不写成已经证明。

3.构建 NEMU 参考库

```
cd $NEMU_HOME
make riscv64-xs-ref_defconfig
make -j$(nproc)
ls -l build/riscv64-nemu-interpreter-so
```

NEMU 的配置里我们重点确认了这几项：

```
CONFIG_RVV=y
CONFIG_RV_SMRNMI=y
CONFIG_NMIE_INIT=y
# CONFIG_MDT_INIT is not set
CONFIG_TVAL_EX_II=y
```

CONFIG_NMIE_INIT=y 是为了让 NEMU 的 NMIE 复位值和香山一致（香山复位为 1）。MDT_INIT 关闭是为了让 MDT 复位为 0。早期没有对齐时，非法指令异常会走到 RNMI/NMIE=0 的路径，NEMU 的 mcause/mepc 和 RTL 对不

上，这个问题在第三阶段文档里会再写一次。

4 构建香山 emu

正式配置：

```
cd $NOOP_HOME
make emu CONFIG=KunminghuV2MinimalConfig JVM_XMX=12G EMU_THREADS=2 -j2
```

早期为了快，也构建过 MinimalConfig：

```
make emu CONFIG=MinimalConfig JVM_XMX=12G EMU_THREADS=2 -j2
```

MinimalConfig 大概 15 分钟 RTL 生成、30 分钟 C++构建，build/emu 约 71 MB。正式配置的构建时间没有单独计时，工具链和命令是一样的。

5 boot flash

boot-minimal.bin 是 16 字节的启动跳转，不是测试程序本身。测试程序用-i 加载，flash 只负责把核带到程序入口。

```
sha256sum artifacts/boot-minimal.bin
# 期望：fea548655a8b6eecacace83e2f4380a18b8aa83bce964c8aae4cffbe2f5a9724
```

早期我们遇到过 difftest 第 2 条指令 mismatch，原因之一是 emu 和 NEMU 用了不同的默认 flash。后来统一用-F 显式传入同一个文件，问题就消失了。所以后面所有命令都带-F，不依赖默认值。

四、Hello 验证

1 编译 hello

hello 程序在 software/hello_main.c，团队标识和 IP 在 software/hello_identity.h 里。团队标识是工大喵喵队，IP 是 192.168.124.133。

```
cd software
make hello
```

2 运行

```
cd $NOOP_HOME
./build/emu \
-i $HOME/risv/project/software/build/hello.bin \
--diff $NEMU_HOME/build/riscv64-nemu-interpreter-so \
-F $HOME/risv/artifacts/boot-minimal.bin
```

3 实际输出

日志保存在 results/kunminghu-v2-final/hello-team20261106.log，关键几行是：

```
The image is .../software/build/hello.bin
The reference model is .../riscv64-nemu-interpreter-so
hello xiangshan, I am 工大喵喵队, IP address 192.168.124.133
Core 0: HIT GOOD TRAP at pc = 0x80000032
Core-0 instrCnt = 417, cycleCnt = 7,754, IPC = 0.053779
```



图 1 hello 输出和 GOOD TRAP

日志里的输出是工大喵喵队；这次运行的指令数是 417，周期数是 7,754。

五、操作日志

日期	操作	结果
10-23	读赛题、确认 vdot 路线、拉 xs-env	确定在昆明湖 V2 上做自定义向量点积
10-27	在 VM 里跑独立 RTL 单元测试	PASS，69,656 个向量
10-28	构建 NEMU 和香山，跑 CoreMark	统一 flash 后 CoreMark GOOD TRAP
10-28	编译 vadd 测试和 vadd-wave	vadd 40 组通过，采到第一份 FST
10-29	打 vdot 补丁，跑功能和异常测试	vdot 功能通过，VL=0 异常对齐
10-30	写 perf 程序，做性能对照	1.532×/1.540×
10-31	构建正式配置，三项回归，归档	KunminghuV2MinimalConfig 下三项都 GOOD TRAP

整个过程中主要用的命令就是下面这些：

```
# 构建
cd $NEMU_HOME && make riscv64-xs-ref_defconfig && make -j$(nproc)
cd $NOOP_HOME && make emu CONFIG=KunminghuV2MinimalConfig JVM_XMX=12G
EMU_THREADS=2 -j2

# 编译测试程序
cd software && make all hello vadd exception-check perf perf-reverse vdot-wave

# 跑 hello
./build/emu -i ../project/software/build/hello.bin \
--diff $NEMU_HOME/build/riscv64-nemu-interpreter-so \
-F ../artifacts/boot-minimal.bin
```

六、遇到的问题

1.OpenLLC 负位宽

第一次构建 KunminghuV2MinimalConfig 时，Chisel 在 openLLC/LLCParam.scala:102 报 Invalid bit range[hi=-4,lo=0]。当时先换了 MinimalConfig 把流程跑通，后来把 NOOP_HOME/NEMU_HOME 显式设好，再构建就通过了。

我们只做到了“设置后能过”，没有做“设/不设变量”的对照实验，所以不能确定根因就是 NOOP_HOME；这一点如实写在这里。

2.difftest 第 2 条指令 mismatch

自编 NEMU 第一次跑的时候，第 2 条指令就 mismatch。查下来是 emu 和 NEMU 用的默认 flash 不一样。后来统一用-F 传入同一个 16 字节 boot-minimal.bin，短测和 CoreMark 都正常了。

3.NMIE/MDT 不一致

非法指令异常时，NEMU 的 mcause/mepc 是 0，RTL 是 2/0x800001fa。最后定位到 NEMU 的 NMIE 复位值和香山不一样，raise_intr 在 NMIE=0 时提前返回。把 CONFIG_NMIE_INIT=y 打开、MDT_INIT 关掉后，两边就对上了。

4.FST 构建失败和 VM 关机

采 vadd 波形时，默认的 emu 没有编译 FST 跟踪。我们单独构建了一个开

EMU_TRACE=fst 的 emu-fst，第一次报 NoSuchElementException:NOOP_HOME，补上变量后继续。后来又遇到一次 VM 在 C++编译阶段关机/崩溃，保留了已经生成的 RTL 和对象文件，改成单任务、OPT_FAST=-O0 续编才完成。

这一段比较折腾，但也让我们后面采 vdot 波形时知道要先把环境变量和编译产物固定好。

七、本阶段结果

香山和 NEMU 的版本、配置、flash 哈希都固定下来了；

hello 程序在昆明湖 V2 上跑通，输出符合要求，HELLO_EXIT=0；

部署和操作过程有完整日志，后面 vadd 波形、vdot 集成、性能测试都用这套环境；

踩过的坑（OpenLLC、flash、NMIE/MDT、FST 构建）都记录下来了，复现时可以直接对照。